

Competitive Programming

Section 3 Editorial: Counting, frequencies, and maps

Competitive Programming Course

2026-07-04

Table of contents

1	How to read this editorial	3
2	Problem A: Value Counts	4
2.1	Restatement	4
2.2	Important constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness	5
2.7	Complexity	5
2.8	C++ solution	6
2.9	Python solution	6
2.10	Common wrong answers	7
2.11	Tests that matter	7
3	Problem B: Frequency Winner	8
3.1	Restatement	8
3.2	Important constraints and traps	8
3.3	Observations	8
3.4	Sample walk-through	8
3.5	Algorithm	9
3.6	Correctness	9
3.7	Complexity	10
3.8	C++ solution	10
3.9	Python solution	10
3.10	Common wrong answers	11
3.11	Tests that matter	11
4	Problem C: Pair Sum Count	12
4.1	Restatement	12
4.2	Important constraints and traps	12
4.3	Observations	12
4.4	Sample walk-through	12
4.5	Algorithm	13
4.6	Correctness	13
4.7	Complexity	14
4.8	C++ solution	14
4.9	Python solution	15
4.10	Common wrong answers	15
4.11	Tests that matter	16
5	Stress testing strategy	17
6	What to carry forward	18

1 How to read this editorial

Frequency tables are simple, but they introduce two habits that matter in much harder problems:

1. summarize the input into a data structure that answers the needed question;
2. define exactly what is being counted.

Most mistakes in this section are not syntax mistakes. They are counting mistakes: missing absent values, breaking ties incorrectly, or counting the same pair twice.

2 Problem A: Value Counts

2.1 Restatement

For each test case, read an array and answer queries of the form:

```
how many times does x appear?
```

Each query is independent. The array does not change.

2.2 Important constraints and traps

The total $n + q$ over all tests is at most 200000, so an $O(n + q)$ solution is expected.

Values fit in signed 32-bit integers, which means they may be negative or as large as 10^9 . A frequency array indexed directly by value is not a safe default. A map/dictionary is the natural representation.

The important trap is absent values: if x does not appear, the answer is 0.

2.3 Observations

If we scan the whole array for every query, the time is $O(nq)$.

Instead, build a frequency table:

```
freq[value] = number of times value appears
```

Then each query is just a lookup.

2.4 Sample walk-through

Array:

```
1 2 2 3 1
```

Build the table:

value	count
1	2
2	2

value	count
3	1

Queries:

query	lookup	output
1	<code>freq[1]</code>	2
2	<code>freq[2]</code>	2
4	absent	0
3	<code>freq[3]</code>	1

2.5 Algorithm

```

for each test case:
    read n and q
    freq = empty map
    repeat n times:
        read value
        freq[value] += 1
    repeat q times:
        read x
        print freq[x] if present, otherwise 0

```

2.6 Correctness

For a fixed test case, consider the array-processing loop. It maintains the invariant:

after processing the first i array values, `freq[v]` equals the number of occurrences of value v among those first i values.

Initially no values have been processed and all counts are zero. When the next value a_i is read, the algorithm increments exactly `freq[a_i]` and leaves all other counts unchanged. Therefore the invariant remains true.

After all n values are processed, the invariant says `freq[v]` equals the number of occurrences of v in the full array. Each query asks for exactly that number. If a queried value is absent from the map, it appears zero times. Thus every printed answer is correct.

2.7 Complexity

Building the table is $O(n)$ expected time with a hash map. Each query is $O(1)$ expected time. The total time is $O(n + q)$ per test case, and memory is $O(d)$, where d is the number of distinct values.

2.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        unordered_map<int, int> freq;
        freq.reserve(2 * n + 1);
        for (int i = 0; i < n; ++i) {
            int x;
            cin >> x;
            ++freq[x];
        }
        while (q--) {
            int x;
            cin >> x;
            auto it = freq.find(x);
            cout << (it == freq.end() ? 0 : it->second) << '\n';
        }
    }
    return 0;
}
```

2.9 Python solution

```
from collections import Counter
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    q = data[idx + 1]
    idx += 2
    freq = Counter(data[idx:idx + n])
    idx += n
    for _ in range(q):
        x = data[idx]
```

```
    idx += 1
    out.append(str(freq.get(x, 0)))
sys.stdout.write("\n".join(out))
```

2.10 Common wrong answers

- Allocating an array of size 10^9 .
- Forgetting that values may be negative.
- Printing nothing for absent values.
- Rebuilding the frequency table for every query.
- Accidentally reading the queries as part of the array because of index mistakes.

2.11 Tests that matter

```
all values equal
all values distinct
negative values
queries for absent values
large values such as  $10^9$ 
many queries after a small array
```

3 Problem B: Frequency Winner

3.1 Restatement

For each array, print:

```
value frequency
```

where `value` is the most frequent array value. If multiple values have the same maximum frequency, choose the smallest value.

3.2 Important constraints and traps

The array may contain negative values. The tie rule is based on numeric value, not first appearance.

Hash-map iteration order is not a tie rule. You must compare candidates explicitly.

3.3 Observations

First count every value. Then scan the frequency table and keep the best candidate.

A candidate (`value`, `count`) is better than the current best if:

```
count > best_count
or
count == best_count and value < best_value
```

This comparison exactly matches the statement.

3.4 Sample walk-through

Array:

```
-1 -2 -2 -1 -3 -3
```

Frequency table:

value	count
-1	2

value	count
-2	2
-3	2

All counts tie. The smallest value is -3, so the output is:

```
-3 2
```

Another example:

```
5 4 3
```

Every value appears once. The smallest value is 3, so the answer is:

```
3 1
```

3.5 Algorithm

```
for each test case:
    build freq
    best_value = any array value
    best_count = -1
    for value, count in freq:
        if count > best_count:
            update best
        else if count == best_count and value < best_value:
            update best
    print best_value, best_count
```

3.6 Correctness

After the frequency table is built, `freq[v]` is the number of occurrences of each value `v`.

The scan over the frequency table maintains this invariant:

after examining some table entries, `(best_value, best_count)` is the correct answer among only the examined values.

Initially no real entry has been accepted, so `best_count = -1` makes the first examined entry become the best. When a new entry `(value, count)` is examined, there are three cases:

- if `count` is larger than `best_count`, the new value is more frequent and must become best;
- if `count` is equal and `value` is smaller, the tie rule chooses the new value;
- otherwise the old best remains better.

Thus the invariant is preserved. After all distinct values are examined, the invariant says the stored candidate is the correct answer for the whole array.

3.7 Complexity

Counting takes $O(n)$ expected time. Scanning distinct values takes $O(d)$, where $d \leq n$. Memory is $O(d)$.

3.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        unordered_map<int, int> freq;
        freq.reserve(2 * n + 1);
        for (int i = 0; i < n; ++i) {
            int x;
            cin >> x;
            ++freq[x];
        }

        int best_value = 0;
        int best_count = -1;
        for (auto [value, count] : freq) {
            if (count > best_count || (count == best_count && value < best_value)) {
                best_value = value;
                best_count = count;
            }
        }
        cout << best_value << ' ' << best_count << '\n';
    }
    return 0;
}
```

3.9 Python solution

```
from collections import Counter
import sys
```

```

def winner(values: list[int]) -> tuple[int, int]:
    freq = Counter(values)
    best_value = values[0]
    best_count = -1
    for value, count in freq.items():
        if count > best_count or (count == best_count and value < best_value):
            best_value = value
            best_count = count
    return best_value, best_count

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    idx += 1
    values = data[idx:idx + n]
    idx += n
    value, count = winner(values)
    out.append(f"{value} {count}")
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Choosing the first value that reaches the maximum frequency.
- Choosing the largest tied value.
- Sorting the original array but forgetting to handle the final run.
- Assuming `unordered_map` iteration order is sorted.
- Initializing `best_value` incorrectly and mishandling all-negative arrays.

3.11 Tests that matter

```

single element
all equal
all distinct
negative tie
tie between negative and positive values
large absolute values

```

4 Problem C: Pair Sum Count

4.1 Restatement

For each array and target k , count index pairs (i, j) such that:

```
i < j  
ai + aj = k
```

The pair is a pair of indices, not just a pair of values. If the same value appears many times, it can create many pairs.

4.2 Important constraints and traps

The total n is up to 200000, so $O(n^2)$ is too slow.

The answer may be as large as $n \text{ choose } 2$, which exceeds 32-bit range. In C++, use `long long` for the answer and frequency counts.

4.3 Observations

Process values from left to right. When the current value is x , any valid previous partner must have value:

```
k - x
```

If that complement appeared c times earlier, the current index forms c new pairs. Then add the current value to the table for future positions.

The order of these two operations matters. Count complements before inserting the current value. That prevents pairing an element with itself.

4.4 Sample walk-through

Array:

```
1 4 2 3 5
```

Target:

5

Trace:

current	complement	seen complement	count	new pairs	answer
1	4		0	0	0
4	1		1	1	1
2	3		0	0	1
3	2		1	1	2
5	0		0	0	2

The answer is 2.

For repeated values:

```
array = 1 1 1 1
target = 2
```

The new pair counts are:

0, 1, 2, 3

so the total is 6.

4.5 Algorithm

```
for each test case:
    seen = empty map
    answer = 0
    for value in array:
        answer += seen[target - value]
        seen[value] += 1
    print answer
```

4.6 Correctness

Before processing position j , maintain this invariant:

$\text{seen}[v]$ is the number of indices $i < j$ with $a_i = v$, and answer is the number of valid pairs (i, r) with $i < r < j$.

Initially $j = 0$, no indices have been processed, all seen counts are zero, and there are no valid pairs in the empty prefix.

At position j , let $a_j = x$. A new valid pair ending at j must use an earlier index $i < j$ with:

$$a_i + x = k.$$

Equivalently:

$$a_i = k - x.$$

By the invariant, there are exactly `seen[k - x]` such earlier indices. The algorithm adds exactly that number to `answer`, so all new valid pairs ending at j are counted once. Then it increments `seen[x]`, making the seen table correct for the next position.

Thus the invariant is preserved. After all positions are processed, every valid pair has some right endpoint j and was counted exactly when that endpoint was processed. Therefore the printed answer is correct.

4.7 Complexity

The algorithm performs one hash-map lookup and one update per array element. Expected time is $O(n)$, and memory is $O(d)$, where d is the number of distinct values.

4.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, target;
        cin >> n >> target;
        unordered_map<int, long long> seen;
        seen.reserve(2 * n + 1);
        long long answer = 0;
        for (int i = 0; i < n; ++i) {
            int value;
            cin >> value;
            auto it = seen.find(target - value);
            if (it != seen.end()) {
                answer += it->second;
            }
        }
    }
}
```

```

        ++seen[value];
    }
    cout << answer << '\n';
}
return 0;
}

```

4.9 Python solution

```

from collections import defaultdict
import sys

def count_pairs(values: list[int], target: int) -> int:
    seen = defaultdict(int)
    answer = 0
    for value in values:
        answer += seen[target - value]
        seen[value] += 1
    return answer

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    target = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(count_pairs(values, target)))
sys.stdout.write("\n".join(out))

```

4.10 Common wrong answers

- Inserting the current value before counting complements, which can count (i, i) when $2 \cdot a_i = k$.
- Counting value pairs instead of index pairs.
- Dividing by two after using the left-to-right method.
- Using `int` for the answer in C++.
- Forgetting negative values and negative targets.

4.11 Tests that matter

```
n = 1: no pairs  
all zeros with target 0  
many repeated equal values  
negative plus positive pairs  
no valid pair  
answer larger than 32-bit
```

5 Stress testing strategy

Frequency problems are easy to stress test because slow reference solutions are short.

For Problem A, the slow reference can scan the array for every query.

For Problem B, the slow reference can sort values and count runs, or use a map and then compare all keys.

For Problem C, the slow reference is the direct double loop:

```
answer = 0
for i from 0 to n - 1:
    for j from i + 1 to n - 1:
        if a[i] + a[j] == k:
            answer += 1
```

That is too slow for full constraints but perfect for random tests with $n \leq 100$. The local generator uses this idea for pair counting.

Good random tests should include:

- small value ranges to create duplicates;
- negative values;
- absent query values;
- all-equal arrays;
- targets that are likely and unlikely.

6 What to carry forward

The main lesson is that counts are often a compressed form of the input. Once you know what must be counted, many problems become a map lookup plus a careful update.

This pattern will return in:

- coordinate compression;
- two-sum and k-sum variants;
- prefix-sum frequency problems;
- graph degree counting;
- combinatorics over repeated values.